

Quick Start

A quick start guide for those that are brand new to Codecov.

This guide is meant to help you get started with Codecov as quickly as possible. By the end of the guide, you will have:

- Integrated Codecov into your CI pipeline
- Uploaded coverage to Codecov
- Viewed coverage reports in Codecov's app as well as in your pull requests / merge requests

Prerequisites

In order to get started, you will need the following:

1. Access to a repository with [GitHub](#), [Bitbucket](#), or [GitLab](#).
 - i. If you are a GitHub user you **MUST** install the [Codecov GitHub app](#) for your organization - Codecov can't function without it.
2. A CI environment that runs tests and generates coverage reports.

Not ready to use Codecov on your own repositories?

Try it out for yourself with the Codecov tutorials for [GitHub](#), [Bitbucket](#), or [GitLab](#) to see what Codecov has to offer.

Getting Started

Step 1: Sign up for Codecov

Sign up for a Codecov account to connect your individual code host account with Codecov. You will be brought to a page with a list of repositories.

codecov

Repos Analytics Members Plan Settings

All Configured **Not Configured** Search Can't find your repo? [Resync](#)

Name	Configure
ruby-standard-2	Configure
example-go	Configure
codecov-demo	Configure
opentelem-python	Configure
example-java-maven	Configure
swift-standard	Configure
example-c	Configure
example-java-gradle	Configure
impact-analysis-example-python	Configure
java-standard	Configure
typescript-standard	Configure
go-standard	Configure
example-julia	Configure
example-rust	Configure

Not seeing any repositories? Try clicking **Not Configured** on the left toggle. If you're a GitHub user, you may need to allow additional repositories to access Codecov in the [app configuration](#). Note that if this is your first time logging into Codecov, it may take a few moments for your repositories to sync and appear in the UI.

github.com/organizations/codecov/settings/installations/1271

OAuth application policy

GitHub Apps (Beta)

Personal access tokens

Integrations

Scheduled reminders

Archive

Logs

Deleted repositories

Developer settings

✓ Read and write access to checks, commit statuses, and pull requests

Repository access

☐ All repositories
This applies to all current and future repositories owned by the resource owner. Also includes public repositories (read-only).

☒ Only select repositories
Select at least one repository. Also includes public repositories (read-only).

Select repositories

Save Cancel

Additional Admin Approval for GitHub users

If you are signing up via GitHub and not an admin of your Github organization, you may need to request access from an admin to authorize Codecov as a third-party GitHub application. For more information see [GitHub Admin Authorization](#).

Step 2: Get the repository upload token

Click on `setup repo` for the repository you would like to use Codecov. This should take you to the setup screen for the repository.

codecov / codecov-demo



Coverage Commits Pulls Settings

Let's get your repo covered

Before integrating with Codecov, ensure that your project already generates coverage reports. Codecov relies on these reports to provide the coverage analysis.

[Read our documentation](#)

Select a setup option

Using GitHub Actions ☒

Using Circle CI ☐

Using Codecov's CLI ☐

Step 1: add global token as repository secret

Admin required to access repo settings > secrets and variable > actions

CODECOV_TOKEN



ca4a16fb-cb5b-4e72-baba-890d2a6bc72b



Step 2: add Codecov to your GitHub Actions workflow yaml file

After tests run, this will upload your coverage report to Codecov:

```
- name: Upload coverage reports to Codecov
  uses: codecov/codecov-action@v4.0.1
  with:
    token: ${{ secrets.CODECOV_TOKEN }}
    slug: codecov/codecov-demo
```



See an example [repo here](#) and [our CLI](#).

Step 3: merge to main or your preferred feature branch

Once merged to your default branch, subsequent pull requests will have Codecov checks and comments. Additionally, you'll find your repo coverage dashboard here. If you have merged, try reloading the page.

How was your setup experience? Let us know in [this issue](#).

Copy the token as shown in step one for later. It will be used to authenticate and verify coverage reports uploaded for this repository.

Step 3: Setup integration/team bot (GitLab and Bitbucket only)

To communicate between your code host and Codecov, you will need to [add a Team Bot](#) for Bitbucket and GitLab.

No further action is required for GitHub users at this step. GitHub utilizes Codecov's GitHub app for this functionality and is handled in Step 1: Signing up for Codecov.

Step 4: Upload coverage reports to Codecov

Use the [Codecov CLI](#) and the repository upload token to upload your coverage report(s) to Codecov. **These tools should be added as a step in your CI pipeline.**

The following CI providers have built-in utilities that will also verify the uploader integrity before uploading to Codecov:

- [Bitrise](#)
- [CircleCI](#)
- [GitHub Actions](#)

We **highly recommend** using these built-in Codecov utilities in their respective workflows.

An example of a workflow using the CLI with GitHub Actions is shown below.

```
.github/workflows/test.yml
```

```
name: API workflow
```

```
on: [push, pull_request]
```

```
jobs:
```

```
  build:
```

```
    runs-on: ubuntu-latest
```

```
    name: Test python API
```

```
    steps:
```

```
      - uses: actions/checkout@v4
```

```
      - name: Install requirements
```

```
        run: pip install -r requirements.txt
```

```
      - name: Run tests and collect coverage
```

```
        run: pytest --cov .
```

```
      - name: Upload coverage reports to Codecov
```

```
        run: |
```

```
          # Replace `linux` below with the appropriate OS
```

```
          # Options are `alpine`, `linux`, `macos`, `windows`
```

```
          curl -Os https://cli.codecov.io/latest/linux/codecov
```

```
          chmod +x codecov
```

```
          ./codecov --verbose upload-process --fail-on-error -t ${
```

```
secrets.CODECOV_TOKEN }} -n 'service'-${{ github.run_id }} -F service -f coverage-  
service.xml
```

You can check out some of our [example repositories](#) for more help on this step.

Step 5: Get coverage analysis from Codecov

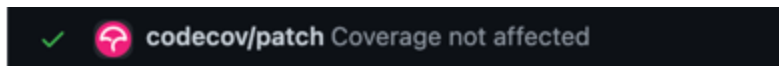
After making the above edits, commit your changes and run your CI pipeline. You should see something similar in the build logs

```

1 ▶ Run make test_env.upload CODECOV_UPLOAD_TOKEN=*** CODECOV_URL=***
6 docker-compose exec api make test_env.container_upload CODECOV_UPLOAD_TOKEN=*** CODECOV_URL=***
7 codecovcli -u *** upload-process --flag unit-latest-uploader --flag unit \
8 --coverage-files-search-exclude-folder=graphql_api/types/** \
9 --coverage-files-search-exclude-folder=api/internal/tests/unit/views/cassetes/**
10 info - 2024-04-29 16:27:03,499 -- ci service found: github-actions
11 info - 2024-04-29 16:27:03,700 -- Process Commit creating complete
12 info - 2024-04-29 16:27:03,852 -- Process Report creating complete
13 info - 2024-04-29 16:27:03,852 -- Finished creating report successfully --- {"response": "{\"external_id\":\"53ac3d75-d4a9-4411-a837-799a78eb8d21\", \"created_at\":\"2024-04-29T16:08:01.470934Z\", \"commit_sha\":\"9d7075665f2ff5a1885ae2056df4af0904ddd3b\", \"code\":null}"}
14 warning - 2024-04-29 16:27:03,853 -- xcrun is not installed or can't be found.
15 warning - 2024-04-29 16:27:03,892 -- No gcov data found.
16 info - 2024-04-29 16:27:03,893 -- Generating coverage.xml report in /app
17
18 info - 2024-04-29 16:27:09,474 -- Found 5 coverage files to report
19 info - 2024-04-29 16:27:09,474 -- > /app/graphql_api/types/enums/coverage_line.graphql
20 info - 2024-04-29 16:27:09,474 -- > /app/unit.coverage.xml
21 info - 2024-04-29 16:27:09,474 -- > /app/coverage.xml
22 info - 2024-04-29 16:27:09,474 -- > /app/api/internal/tests/unit/views/cassetes/test_compare_view/TestCompareDetailsView/test_compare_line_coverage_view.yaml
23 info - 2024-04-29 16:27:09,474 -- > /app/graphql_api/types/coverage_totals/coverage_totals.graphql
24 info - 2024-04-29 16:27:09,902 -- Your upload is now processing. When finished, results will be available at: https://app.codecov.io/github/codecov/codecov-api/commit/9d7075665f2ff5a1885ae2056df4af0904ddd3b
25 info - 2024-04-29 16:27:10,203 -- Process Upload complete
26 docker-compose exec api make test_env.container_upload_test_results CODECOV_UPLOAD_TOKEN=*** CODECOV_URL=***
27 codecovcli -u *** do-upload --report-type "test_results" \
28 --files-search-exclude-folder=graphql_api/types/** \
29 --files-search-exclude-folder=api/internal/tests/unit/views/cassetes/** || true
30 info - 2024-04-29 16:27:10,608 -- ci service found: github-actions
31 info - 2024-04-29 16:27:10,671 -- Found 1 test_results files to report
32 info - 2024-04-29 16:27:10,671 -- > /app/unit.junit.xml
33 info - 2024-04-29 16:27:11,090 -- Process Upload complete

```

In your pull request, you should see a single status checks like



and a pull request comment similar to

Codecov Report

Attention: Patch coverage is 60.00000% with 6 lines in your changes are missing coverage. Please review.

! Current head [eb65b18](#) differs from pull request most recent head [cfffad3d](#). Consider uploading reports for the commit [cfffad3d](#) to get more accurate results

► Additional details and impacted files



By default, Codecov will gate and report on coverage changes on Git diffin the Pull request; with overall project coverage reporting available on the Codecov dashboard.

Step 6: View the dashboard on Codecov

After merging the above pull request, go back to Codecov and view the dashboard of your repository to see trends in coverage over time.

Overview

Flags

Components

Branch Context

main

Coverage on branch

95.98%

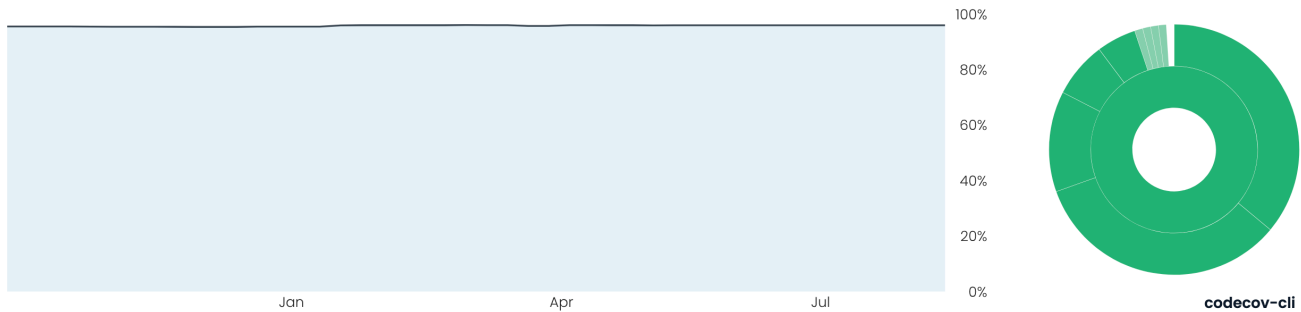
3 Months trend

+0.43%

Source: latest commit 1478514

2823 of 2941 lines covered

Hide charts



Code tree File list codecov-cli

All flags

Search for files

Files	Tracked lines	Covered	Partial	Missed	Coverage %
codecov_cli	2934	2823	0	111	96.22%
setup.py	7	0	0	7	0.00%

You can also view coverage for individual commits and pull requests, as well as coverage overlays to get a better understanding of what code hasn't been tested.

⚠️ Not seeing coverage on the dashboard?

Go to the [Configuration](#) -> [General](#) tab in Codecov and double-check that the default branch is properly set.

Tips and Tricks

1. Want project coverage checks and project coverage report in your PR comments as well? Add the following to your `codecov.yml`

The snippet below adds project coverage checks to every PR.

```
codecov.yml
```

```
coverage:
  status:
    project: #add everything under here, more options at
https://docs.codecov.com/docs/commit-status
    default:
      # basic
```

```
target: auto #default
threshold: 0%
base: auto
```

The snippet below adds project coverage reporting to a PR comment from Codecov

codecov.yml

```
comment:                #this is a top-level key
  layout: " diff, flags, files"
  behavior: default
  require_changes: false # if true: only post the comment if coverage changes
  require_base: false    # [true :: must have a base report to post]
  require_head: true     # [true :: must have a head report to post]
  hide_project_coverage: false # [true :: only show coverage on the git diff aka
patch coverage]
```

2. Not ready to start using blocking status checks? Set them as informational while you and your team get started with code coverage.

codecov.yml

```
coverage:
  status:
    project:
      default:
        informational: true
    patch:
      default:
        informational: true
```

```
# When modifying this file, please validate using
# curl -X POST --data-binary @codecov.yml https://codecov.io/validate
```

3. Check out our [recipe list](#) for common configuration questions.
4. Enforce relative or absolute targets and thresholds during your CI build using the [Codecov YAML](#)
5. Use [Components](#) to split up a repository into separate projects by file path. This is great for monorepos or projects that include several teams or components.
6. Use [flags](#) to categorize coverage results in a single repository. This is great for monorepos or projects that include different types of test coverage (e.g., unit, end-to-end, integration).
7. Use Codecov with the [Chrome Browser Extension](#) or [Firefox Browser Extension](#) (GitHub Only for now).

 Updated 5 months ago

[GitHub Tutorial](#) →

Did this page help you?  Yes  No